ASIAN INSTITUTE OF TECHNOLOGY

Faculty of Advanced Science and Technology

A7: MCP-Server, AI Agent, and External Tool Integration

NATURAL LANGUAGE UNDERSTANDING (NLU)

SEMESTER: JANUARY 2026

Submitted By:
Tisa Bajracharya (st126686)

Submitted To:

Dr. Chaklam Silpasuwanchai

Mr. Todsavad Tangtortan

March 2026

Table of Contents

Table of Figures

This report documents the implementation of an integrated AI Agent ecosystem using the Model Context Protocol (MCP). The system was built using n8n deployed locally via Docker, exposed to the internet using ngrok, and connected to both Telegram and Google Calendar for real-world task automation.

Task 1: MCP Infrastructure and Server Setup

1.1 Server Deployment

The n8n environment was deployed locally using Docker with a PostgreSQL database for persistent storage. The provided docker-compose.yaml file was used as the base configuration. A .env file was created to store database credentials and the ngrok URL separately from the main configuration.

The following tools and configuration were used:

- Docker Desktop running on Windows
- docker-compose.yaml with simple memory and n8n latest image
- .env file containing DB_USER, DB_PASSWORD, DB_NAME, and NGROK_URL
- ngrok free tier used to expose port 5678 to the internet

The docker-compose setup was started using the following command:

```
docker compose up -d
```

The ngrok tunnel was started using:

```
ngrok http 5678
```

The public ngrok URL obtained was:

```
https://ben-inappeasable-semisolemnly.ngrok-free.dev
```

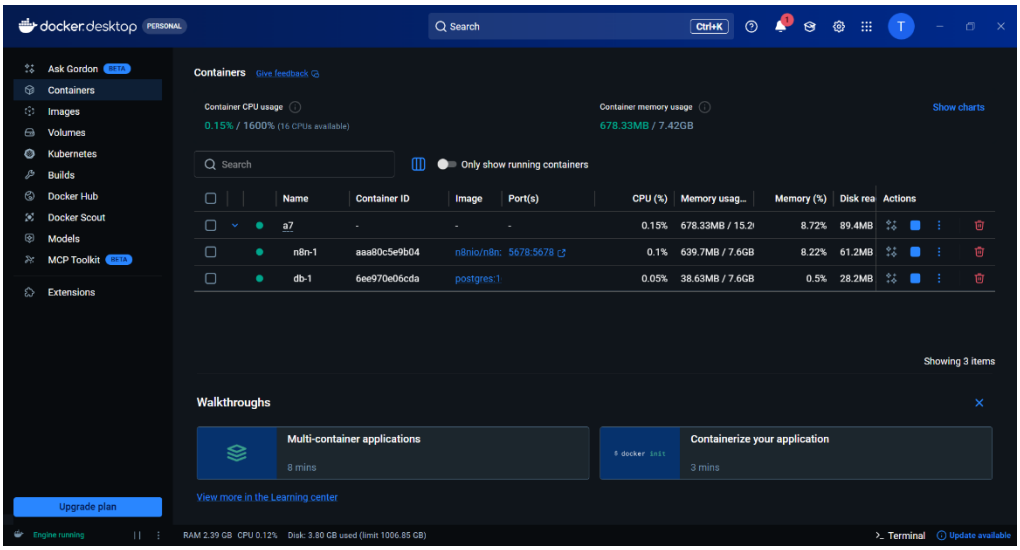

Figure 1: Docker desktop

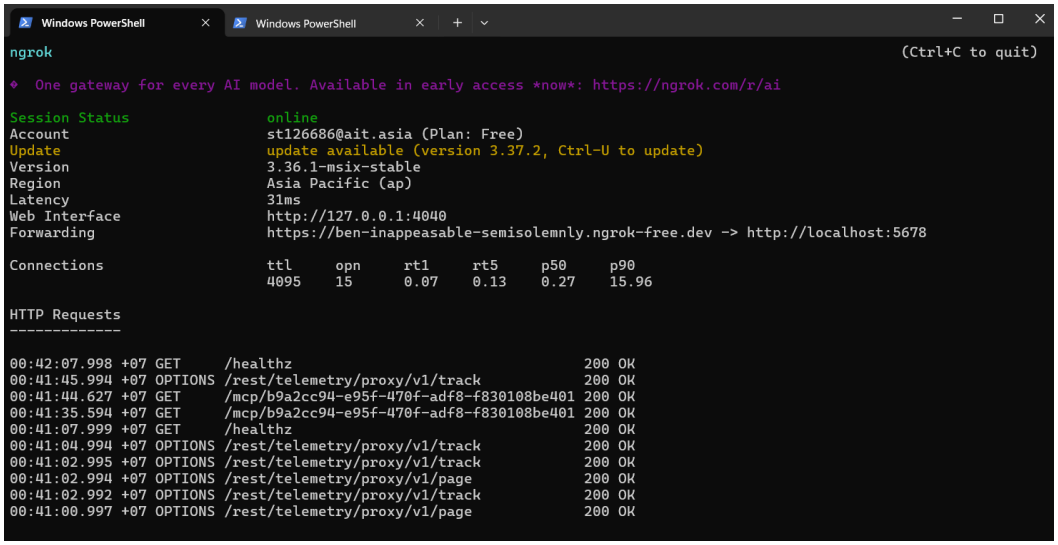

Figure 2:ngrok terminal

1.2 MCP Server Workflow

A dedicated n8n workflow named 'MCP Server' was created to act as the MCP Server. This workflow uses an MCP Server Trigger node as the entry point, with three tools connected to it via dotted tool connections.

The three tools implemented are:

- Calculator: Allows the AI Agent to perform mathematical calculations
- Date and Time: Allows the AI Agent to retrieve the current date and time
- Code Tool: Allows the AI Agent to run JavaScript code for text formatting and manipulation

After building the workflow, it was published using the Publish button in n8n. The production MCP URL generated was:

> https://ben-inappeasable-semisolemnly.ngrok-free.dev/mcp/b9a2cc94-e95f-470f-adf8-f830108be401

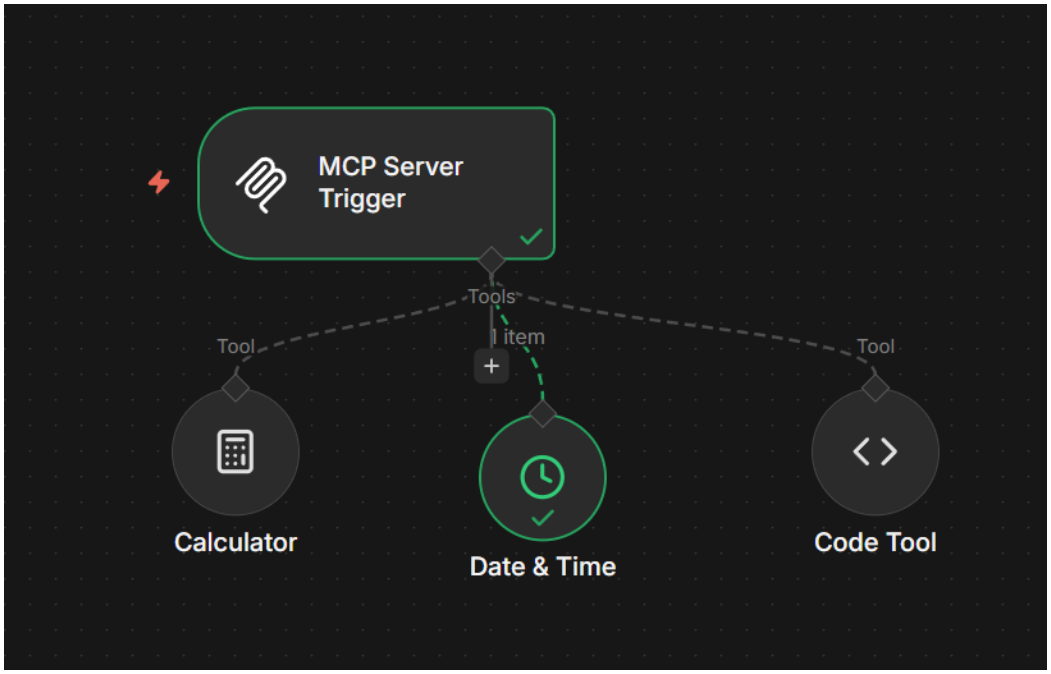

Figure 3:n8n MCP Server workflow with MCP Server Trigger

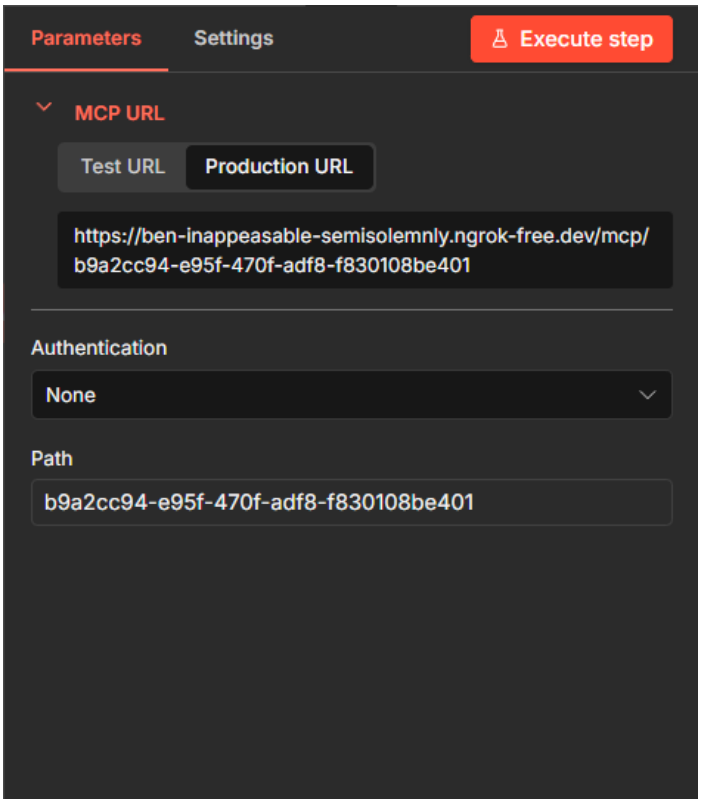

Figure 4:MCP Server Trigger node panel Production URL

1.3 AI Agent Client

A separate n8n workflow named 'AI Agent' was created to act as the AI Agent client. This workflow uses a chat trigger as the entry point and connects to the Groq API for its language model.

The AI Agent was configured with the following components:

Table 1: AI Agent Component

Component	Configuration
Trigger	When chat message received
Language Model	Groq Chat Model (llama-3.3-70b-versatile)
Memory	Simple Memory
Tool	MCP Client connected to MCP Server Production URL

The MCP Client was configured with the SSE server transport type and the production URL of the MCP Server workflow. This allows the AI Agent to discover and use all three tools registered in the MCP Server.

The AI Agent was verified to be working correctly by opening the n8n chat interface and asking 'What time is it right now?' The agent successfully used the Date and Time tool from the MCP Server to return the correct time.

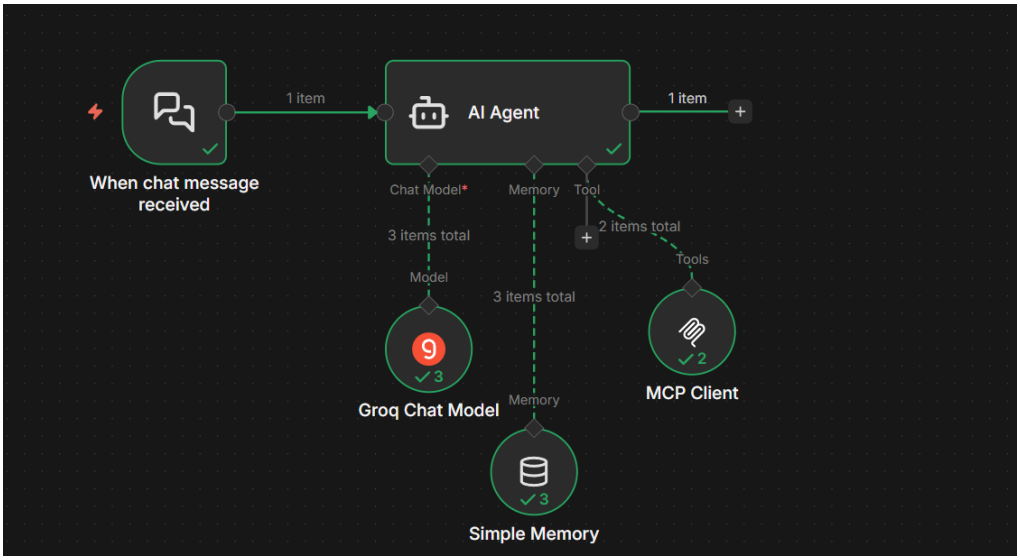

Figure 5: AI Agent workflow

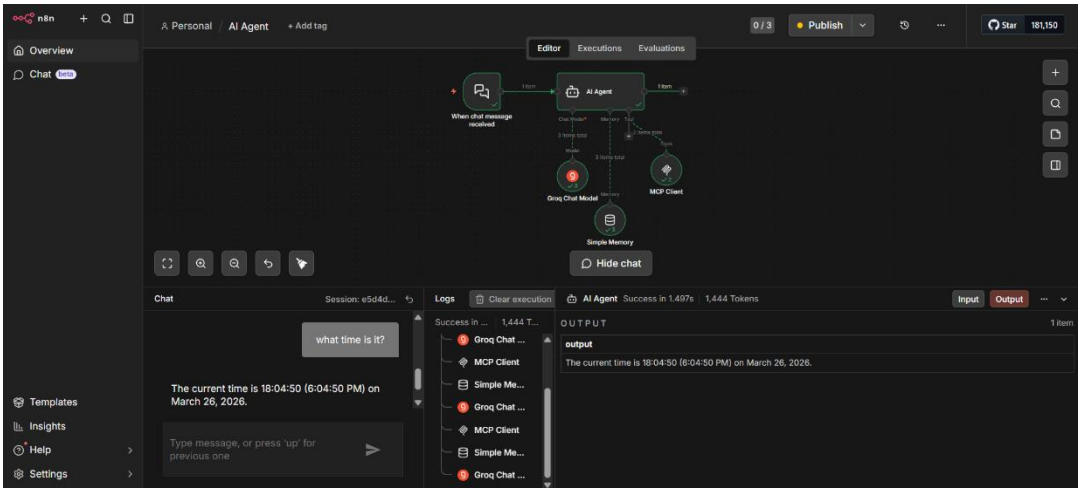

Figure 6: n8n chat interface asking the question 'What time is it right now?' and the agent's correct response using the MCP Date and Time tool

Task 2: Telegram and Google Calendar Integration

2.1 Telegram Bot API

A Telegram bot named 'tizz99bot' was created using BotFather on Telegram. The bot was then connected to the AI Agent workflow in n8n using a Telegram Trigger node configured to listen for incoming messages.

The following steps were completed to set up the Telegram integration:

- Created a new Telegram bot via BotFather using the /newbot command
- Obtained the bot API token from BotFather
- Added a Telegram Trigger node (On message) to the AI Agent workflow
- Created a Telegram credential in n8n using the bot token
- Added a 'Send a text message' Telegram node to send replies back to the user
- Registered the webhook with Telegram using the ngrok URL so messages are forwarded to n8n

The Telegram Send node was configured to dynamically use the chat ID from the incoming message and the AI Agent's output as the reply text. The Postgres Chat Memory session ID was also configured to use the Telegram chat ID to maintain conversation context per user.

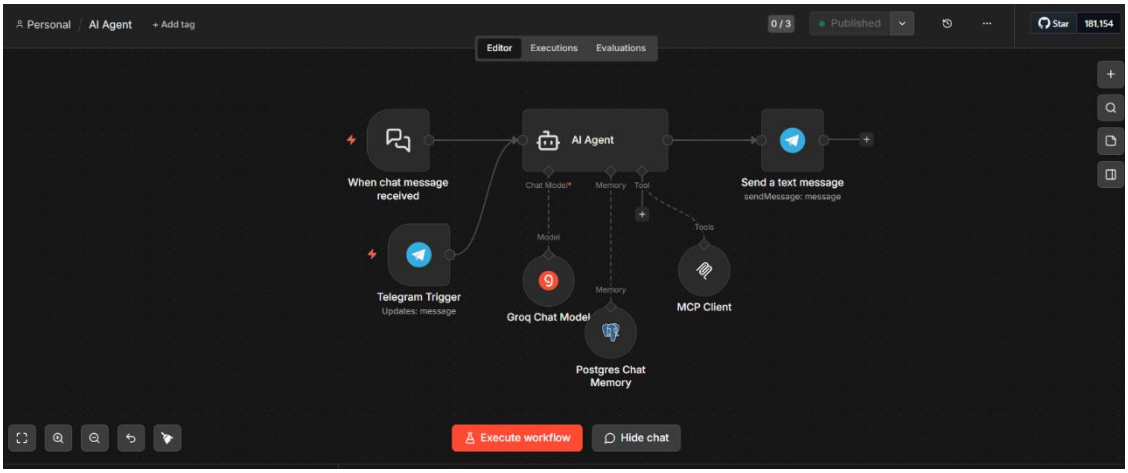

Figure 7: n8n AI Agent workflow connected to Telegram Trigger and Telegram Send nodes

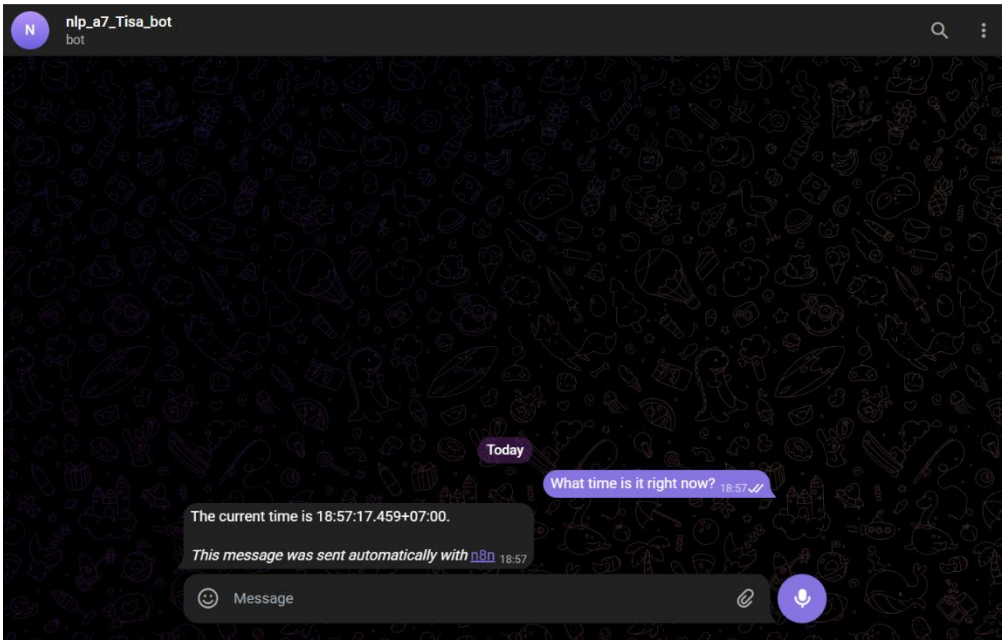

Figure 8: Telegram chat with a test message sent to nlp_a7_Tisa_bot and the bot's reply

2.2 Google Calendar Tool

The Google Calendar tool was integrated into the AI Agent workflow to allow the agent to create, read, and manage calendar events. A Google Cloud project named 'n8n-calendar' was created and the Google Calendar API was enabled.

The OAuth2 credentials were set up as follows:

- Created a new Google Cloud project: n8n-calendar
- Enabled the Google Calendar API
- Configured the OAuth consent screen with External user type
- Created an OAuth 2.0 Client ID (Web application type)
- Added the n8n ngrok callback URL as an Authorized Redirect URI
- Added the Google account as a test user
- Connected the Google Calendar credential in n8n via OAuth2

The Google Calendar node was configured with the following additional fields to ensure events are created correctly:

- Summary: set using $fromAI('summary', 'The title of the calendar event')
- Start time: set using $fromAI('startTime', 'The start time of the event in ISO format')
- End time: set using $fromAI('endTime', 'The end time of the event in ISO format')

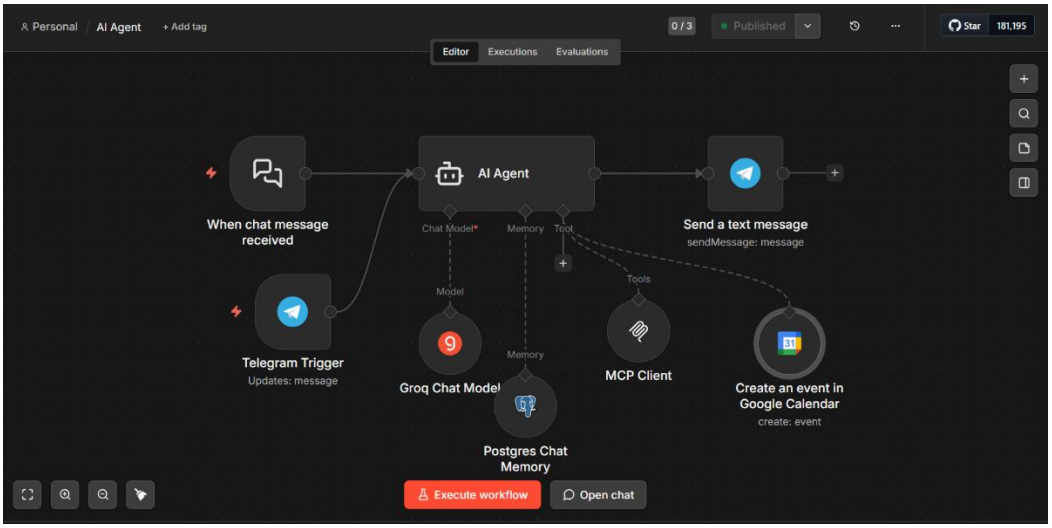

Figure 9: n8n AI Agent workflow with the Google Calendar tool node connected to the AI Agent

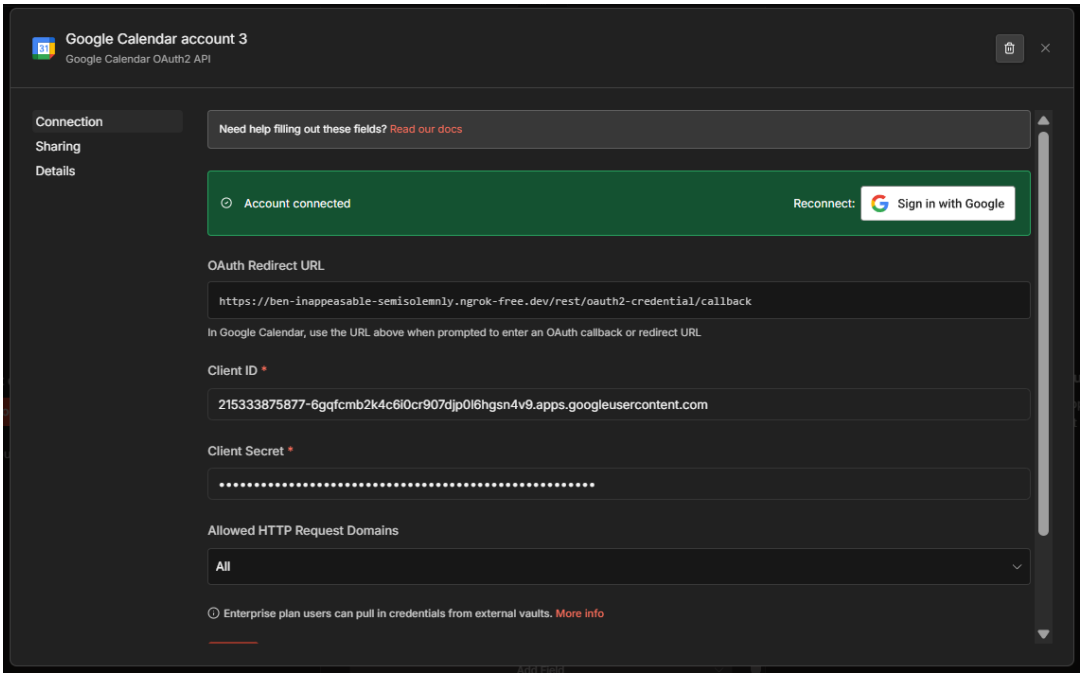

Figure 10: Google Calendar node configuration panel with the credentials

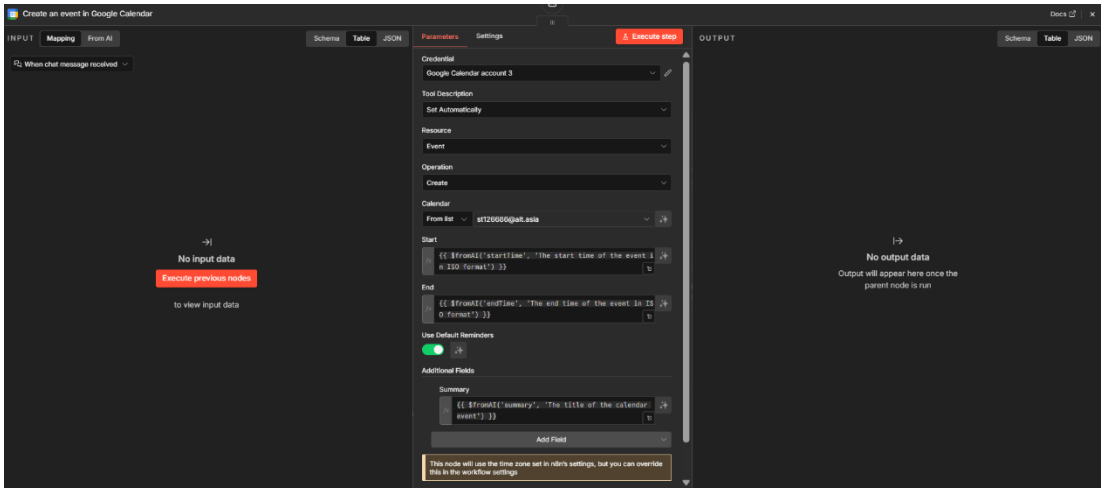

Figure 11: Google Calendar node configuration panel field settings

2.3 Automated Project Scheduling

The AI Agent was tested by sending a project scheduling command via Telegram. The agent was instructed to create four project phase events in Google Calendar with specific dates and times.

The following command was sent via Telegram:

> Please create 4 Google Calendar events for my NLP project:
>
> 1. Literature Review on April 1, 2026 from 9am to 11am
>
> 2. Project Proposal on April 8, 2026 from 9am to 11am
>
> 3. Update Progress on April 15, 2026 from 9am to 11am
>
> 4. Final Presentation on April 22, 2026 from 9am to 11am

The AI Agent successfully created all four events in Google Calendar. The four project phases created were:

Table 2: Events created in Google Calendar by AI Agent

Phase	Event Details
1st Phase	Literature Review - April 1, 2026, 9:00 AM to 11:00 AM
2nd Phase	Project Proposal - April 8, 2026, 9:00 AM to 11:00 AM
3rd Phase	Update Progress - April 15, 2026, 9:00 AM to 11:00 AM
4th Phase	Final Presentation - April 22, 2026, 9:00 AM to 11:00 AM

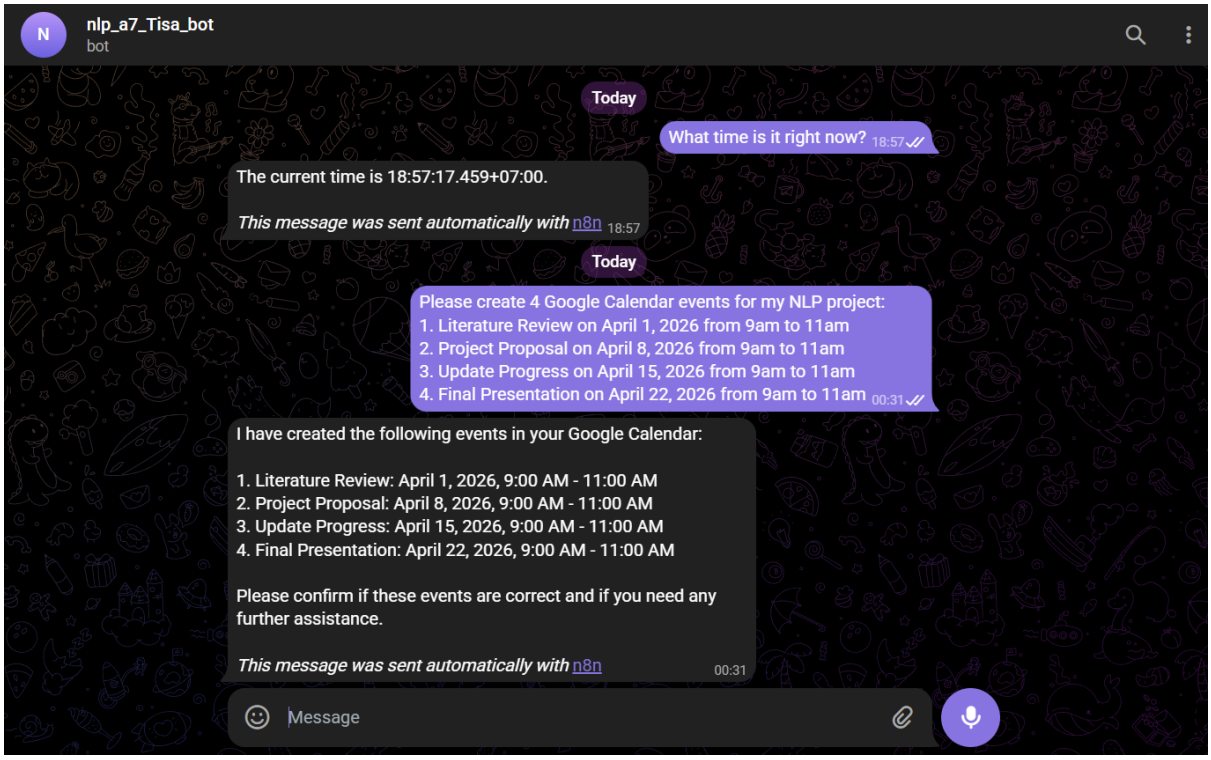

Figure 12: Telegram conversation to schedule events in Google Calendar

2.4 Interaction Verification

To verify the integration, the bot was asked via Telegram to confirm the project phases that were scheduled. The AI Agent successfully retrieved and reported the events from Google Calendar, demonstrating that it can both write to and read from the calendar.

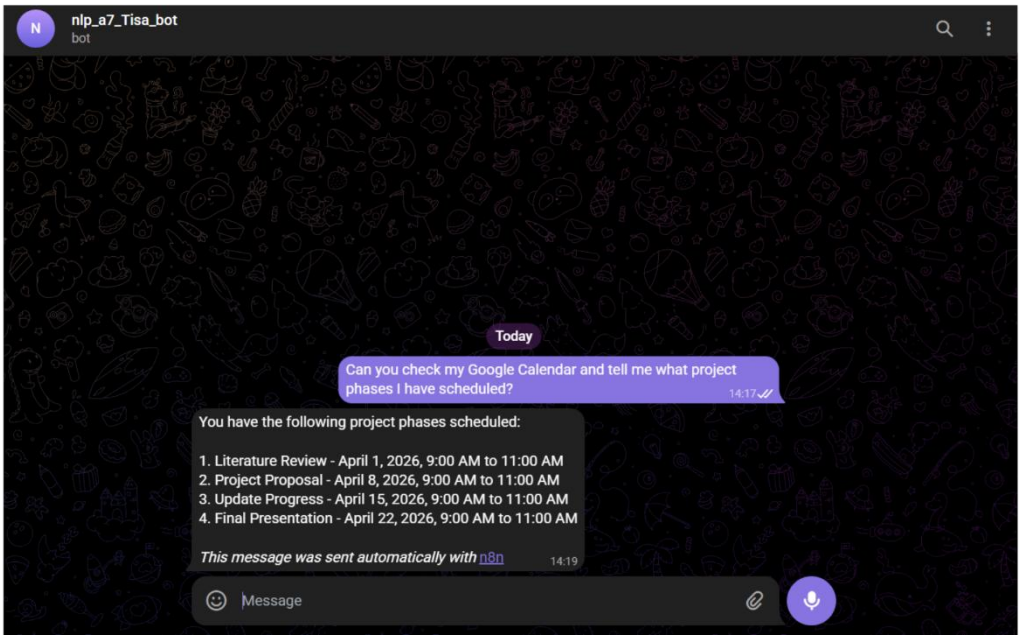

Figure 13: Telegram conversation verifying scheduled events in Google Calendar

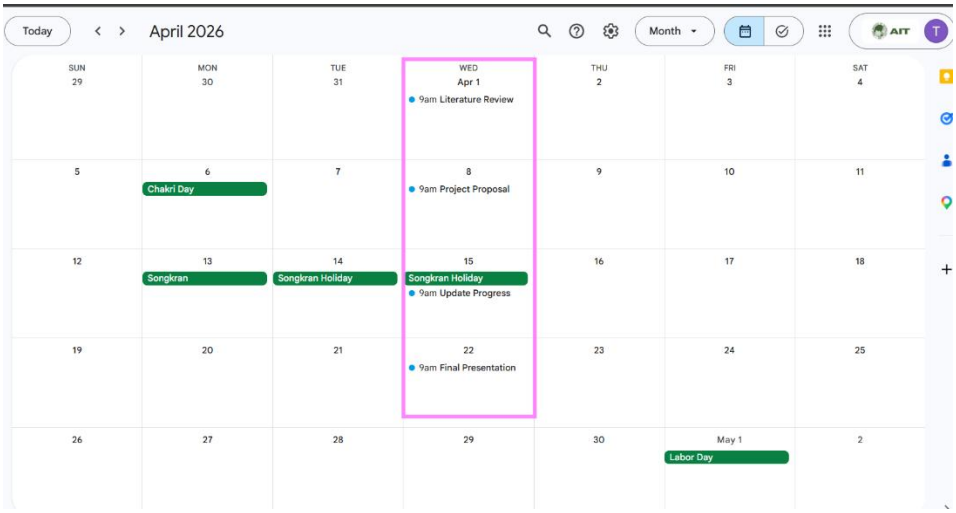

Figure 14: Google Calendar view showing all 4 project phase events created by the AI Agent

Summary

This task demonstrated how modern AI tools can be connected together to create a practical, real-world automation system. By deploying n8n locally with Docker and exposing it through ngrok, a fully functional AI Agent was built that can receive natural language commands via Telegram and take meaningful actions such as creating and reading Google Calendar events.

The Model Context Protocol proved to be an effective way to give the AI Agent access to external tools in a modular and scalable way. Rather than hardcoding specific capabilities, the agent can dynamically discover and use any tools registered in the MCP Server, making the system easy to extend in the future.

The integration of a free language model through Groq showed that powerful AI agents do not require expensive infrastructure. The entire system runs on a local machine and communicates with external services through a simple tunnel, which makes this approach accessible and cost-effective for personal or small-scale use.

Overall, this implementation showed that with the right tools and architecture, it is possible to build an AI agent that understands natural language, manages schedules, and communicates through messaging platforms, all without relying on any cloud-hosted AI infrastructure.